10-Mark Questions & Answers

Q1. Explain the complete DBMS System Architecture with a neat labeled diagram.

Introduction

A DBMS has a layered architecture separating users, query processing, storage management, and physical data. Understanding this architecture helps in knowing how a user's query travels from input to disk and back.

The Three Types of Users

1. Naive Users:

End-us
Exampl

2. Application Programmers:

Write D
Develop

3. Sophisticated Users:

Interact

4. Database Administrator (DBA):

The sup

The Query Processor

The Query Processor translates user queries into actionable operations:

* **DDL Interpreter:** Parses `CREATE`, `ALTER`, `DROP` statements and records schema changes in the Data Dictionary.

* **DM
plan.

The Storage Manager

The Storage Manager interfaces between the DBMS and the physical disk:

* **Authorization & Integrity Manager:** Validates user access rights and enforces data integrity constraints.

* **Tra

Physical Data Structures

The actual data on disk includes:

* **Da

Complete System Architecture Diagram

+----------+

|

Q2. Explain in detail the three-schema architecture and how it achieves data independence.

Introduction

The Three-Schema Architecture (also called ANSI/SPARC Architecture) was proposed to provide data abstraction and data independence. It separates the physical storage, logical design, and user views into three distinct layers, linked by mappings.

The Three Levels in Detail

Level 1 - Internal Level (Physical Schema):

The low

Managed by: System software and DBA.

*Examp

Level 2 - Conceptual Level (Logical Schema):

The mid

Managed by: DBA.

*Examp

Level 3 - External Level (View Level):

The hig
exist sir

Managed by: Application programmers / DBA defines views.

*Examp

The Two Mappings and Data Independence

Mapping 1: External Conceptual

Describ

Mapping 2: Conceptual Internal

Describ

[External View 1] [External View 2]

|

Q3. Compare and contrast DBMS Approach and File System Approach in detail.

File System Approach

The File System approach stores data in a collection of OS-managed files. Each application program creates and manages its own files independently, with no centralized coordination.

DBMS Approach

The DBMS approach stores all data centrally under a dedicated software system that manages storage, access, integrity, and security uniformly for all applications.

Detailed Comparison Table

| Feature | File System | DBMS |

| :--- | :-

When to Use Which?

File System: Suitable for very simple, single-user, standalone applications with minimal data - e.g., a personal notes file.

DBMS: Suitable for all real-world, multi-user, enterprise-grade applications - e.g., banking, hospitals, e-commerce, airlines.

Q4. Explain Data Abstraction and Data Independence in detail with diagrams.

Data Abstraction

Data Abstraction is the process of hiding the complex details of how data is stored and maintained, exposing only the information relevant to the user. The goal is to simplify the user's interaction with the database.

The DBMS provides abstraction at three levels:

Physical Level - *How is data stored?*

Handles this.

Logical Level - *What data is stored?*

Defines

View Level - *What does this user see?*

A clerk security

Data Independence

Logical Data Independence (harder to achieve):

*Chang

If the DBA adds a new column `Address` to `Employee`, existing programs that only use `EmpID` and `EmpName` do not break. Only the External-Conceptual mapping is updated.

Physical Data Independence (easier to achieve):

*Chang

If the DBA switches from a heap file to a clustered B+ tree for the `Employee` table (for faster queries), the SQL schema (`CREATE TABLE`) does not need to change. Only the Conceptual-Internal mapping is updated.

[View A] [View B] <- Protected by Logical Data Independence

| Map

Q5. Describe the different categories of Database Users and the role of the DBA.

The Four Categories of Database Users

1. Naive (Parametric) Users:

The lar
limited t

2. Application Programmers:

Build th

3. Sophisticated Users:

Directly
intellige

4. Specialized Users:

Work w

The Database Administrator (DBA)

The DBA is the most powerful user in the system with complete authority over the database. The DBA's key duties include:

1. Schema Definition: Writes the initial DDL to create all tables, views, constraints, and relationships.

2. Storage Structure and Access Method Definition: Decides the physical organization, indexing strategy, and partitioning to balance performance and storage cost.

3. Schema and Physical Organization Modification: Alters schemas and reorganizes physical storage as business requirements evolve, without disrupting ongoing operations.

4. Granting User Authorizations: Uses `GRANT` and `REVOKE` to control who can read, insert, update, or delete specific data. Implements role-based access control (RBAC).

5. Routine Maintenance: Schedules and performs regular data backups. Tests restore procedures. Monitors disk usage and query performance.

6. Performance Tuning: Analyzes slow queries using execution plans, creates or drops indexes, restructures queries, and adjusts buffer sizes for optimal throughput.

7. Database Security Auditing: Reviews audit logs for suspicious access patterns or policy violations. Implements encryption for sensitive data at rest and in transit.

Q6. Explain the Query Processor and Storage Manager and how a query flows from user to disk.

Overview

When a user submits a SQL query, it travels through multiple layers of the DBMS before the result is returned. This flow involves both the Query Processor and the Storage Manager.

The Query Processor Components

DDL Interpreter:

Handles
and if v

DML Compiler:

Handles

Query Evaluation Engine:

Execute
disk.

Complete Query Flow

Step 1: User types SQL query in client

|

Q7. Discuss the need for a database system over traditional file-based systems.

Introduction

Before DBMS, organizations stored data in operating system files (text files, CSV files). While simple to create, this approach caused numerous serious problems as data grew in size and importance.

Advantages of DBMS / Why Database is Needed

1. Controlling Data Redundancy:

In file s
address
storage

2. Restricting Unauthorized Access:

File sys
access

3. Providing Persistent Storage:

Prograr
the data

4. Storage Structures for Efficient Query Processing:

A DBM
every li

5. Providing Backup and Recovery:

Hard dr
every t
systems

6. Providing Multiple User Interfaces:

Differen
program

7. Representing Complex Relationships:

Real-wo

systems
elegant

8. Enforcing Integrity Constraints:

Data qu
order n
PRIMA

9. Concurrency Control:

When t
conditio

Q8. Explain DBMS architecture types (1-tier, 2-tier, 3-tier) in detail with diagrams, advantages/disadvantages, and use cases.

Introduction

DBMS architecture defines how the various components of a database system are distributed across different machines (tiers/layers). Choosing the right architecture is critical for performance, scalability, and maintainability.

1-Tier Architecture

Definition: All components - user interface, application logic, and database - reside on the same single machine. The user directly interacts with the database without any network.

Diagram:

+--------

Advantages:

* Simp

Disadvantages:

* Not s

Use Case: Personal databases, local development environments, simple standalone tools.

*Examp

2-Tier Architecture

Definition: Splits the system into two tiers - the client (presentation + application logic) and the database server (data

storage). They communicate over a network.

Diagram:

LAN / N

Advantages:

* Supp

Disadvantages:

* Busi

Use Case: Small office networks, LAN-based internal business applications.

*Examp

3-Tier Architecture

Definition: The most modern and widely-used architecture. Splits the system into three distinct and independent layers: Presentation Tier, Application Tier, and Database Tier.

Diagram:

+--------

Advantages:

* High

Disadvantages:

* More

Use Case: All modern web and mobile applications at scale.

*Examp

Summary Comparison

| Feature | 1-Tier | 2-Tier | 3-Tier |

| :--- | :-

Q9. Explain the Entity-Relationship Model in detail - entities, entity sets, relationship sets, attributes - with notations and examples.

Introduction

The Entity-Relationship (ER) Model is a high-level conceptual data model developed by Peter Chen in 1976. It allows database designers to express the logical structure of a database graphically using entities, attributes, and relationships.

---

1. Entities and Entity Sets

* **Entity:** A distinguishable object in the real world (e.g., Student "Alice", Course "DBMS").

* **En

---

2. Attributes

Properties describing an entity set:

* **Sin

---

3. Relationship Sets

An association among entities from two or more entity sets.

* **De

ER Symbol Notation Chart

```
text
```

Entity S

Key Attribute Multivalued Attr Derived Attr

/---------

Relationship Identifying Rel Total Participation

/\

Q10. Explain all types of Attributes with diagrams and real examples.

Introduction

An Attribute is a property or characteristic that describes an entity in an entity set or a relationship in a relationship set.

---

1. Simple vs. Composite Attributes

* **Simple Attribute:** Cannot be sub-divided into smaller components. Example: `Emp_ID`, `Salary`.

* **C

compos

```
text
```

(Addres

---

2. Single-Valued vs. Multivalued Attributes

* **Single-Valued Attribute:** Holds a single value for an entity instance. Example: `Date_of_Birth`, `Gender`.

* **M

Exampl

---

3. Stored vs. Derived Attributes

* **Stored Attribute:** Saved physically on disk. Example: `Date_of_Birth`.

* **D
**Dashe

---

4. Key Attributes vs. Null Attributes

* **Key Attribute:** Uniquely identifies each entity in an entity set. Text is **Underlined**. Example: <u>`Roll_No`</u>.

* **Nu
person

Q11. Explain Mapping Cardinality Constraints in detail (1:1, 1:N, N:1, M:N), including min..max notation and Total/Partial participation.

1. Mapping Cardinalities

```
text
```

1. One-

* **One-to-One (1:1):** An entity in A maps to at most 1 entity in B, and vice versa. Example: `EMPLOYEE` *manages* `DEPARTMENT`.

* **C
`DEPAR

---

2. Min..Max Constraint Notation

Replaces basic cardinalities with explicit structural constraints `(min..max)` placed on participating edges:

* **mir

Example: `EMPLOYEE (0..1) ---- MANAGES ---- (1..1) DEPARTMENT`

* Emp

Q12. Explain Weak and Strong Entity Sets in detail - discriminator, identifying relationship, notation, and a complete example with primary key derivation.

1. Concept

* **Strong Entity Set:** Has a primary key sufficient to uniquely identify all entities.

* **W
Set** vi

---

2. Discriminator (Partial Key)

A weak entity set has a Discriminator (or Partial Key) - an attribute that distinguishes entities belonging to the *same* owner

entity. Represented with a dashed underline.

---

3. Complete Example: `EMPLOYEE` & `DEPENDENT`

```
text
```

+--------

* **Owner Entity:** `EMPLOYEE(Emp_ID, Name, Salary)` -> `PK = Emp_ID`.

* **We

Q13. Explain Keys in the ER model in detail - primary key for entity sets and relationship sets, and key rules for all four binary relationship types.

1. Types of Keys

* **Superkey:** A set of one or more attributes that uniquely identifies an entity.

* **Ca

---

2. Key Rules for Relationship Sets

Let R be a binary relationship set between entity sets A and B:

1. Many-to-Many (M:N):

* Prim

Q14. Explain Specialization and Generalization in detail - overlapping/disjoint, attribute inheritance, single/multiple inheritance (lattice), completeness constraints.

1. Definitions

* **Specialization (Top-Down):** Splitting a superclass into sub-groupings based on specific traits.

* **Ge

---

2. Constraints on Hierarchies

* **Disjointness Constraint:**

* **Dis

---

3. Inheritance & Lattice Structure

* **Attribute Inheritance:** Subclasses inherit all attributes and relationships of the superclass.

* **Sin

Q15. Explain Aggregation in the EER model in detail with the proj_guide/eval_for example - why it's needed and how it's represented.

Why Aggregation is Needed

Standard ER modeling prohibits relationships between relationships. If we need to express a relationship that involves another relationship, standard ER fails or produces incorrect ternary semantics.

Example Scenario

* `STUDENT` works on `PROJECT` via relationship `GUIDED_BY` (with `INSTRUCTOR`).

* Now
PROJE

Diagram & Representation

Aggregation places a bounding box around `[STUDENT, GUIDED_BY, PROJECT]`, treating it as a single higher-level entity set that connects to `EVALUATOR` via `EVALUATES`.

```
text
```

+--------

Q16. Draw and explain a complete ER diagram for a University Database System.

System Requirements

* **Entities:** `STUDENT`, `INSTRUCTOR`, `COURSE`, `DEPARTMENT`, `SECTION`.

* **We

Relational Schema Derived from ER

1. `DEPARTMENT` (<u>Dept_Name</u>, Building, Budget)

2. `INST

Q17. Explain all Constraints in the ER Model - Key Constraint, Participation Constraint, and Mapping Cardinality Constraint.

1. Key Constraint: Specifies that a set of attributes uniquely identifies entities in an entity set or limits an entity to at most 1 relationship instance (1:1 or 1:N).

2. Parti

Q18. Design an ER diagram for the Employee-Department-Project-Dependent scenario.

Entity Sets & Attributes

* `EMPLOYEE` (<u>Emp_ID</u>, Name, Salary, DOB)

* `DEP

Relationships

* `WORKS_IN` (`EMPLOYEE` N : 1 `DEPARTMENT`) [Total participation for Employee]

* `MAN

Q19. Draw a complete ER diagram for the Company Database; convert it fully into a relational schema with keys.

Converted Relational Schema

1. `EMPLOYEE` (<u>Emp_ID</u>, FName, LName, BDate, Address, Sex, Salary, Super_Emp_ID, Dept_ID)

* `Sup

Q20. Explain all eight steps of mapping an ER diagram to a relational model, with one example per step.

* **Step 1: Strong Entity Sets** -> Create relation with all simple attributes.

* **Ste

Q21. Choose two systems (Railway Reservation and Online Food Delivery) - draw ER diagrams and derive relational schemas for both.

System 1: Railway Reservation System

* **Relations:**

1. `PAS

System 2: Online Food Delivery System

* **Relations:**

1. `CUS

Q22. Design a complete ER diagram for the University Database (Student, Course, Instructor, Department, Exam) with a weak entity and multivalued attribute; convert fully to relational schema.

Relational Schema

1. `DEPARTMENT` (<u>Dept_Code</u>, Dept_Name, Building)

2. `INS

Q23. Explain specialization/generalization mapping using EMPLOYEE->SECRETARY/TECHNICIAN/ENGINEER and VEHICLE->CAR/TRUCK - show all four mapping approaches and discuss trade-offs.

Example 1: `EMPLOYEE` -> `SECRETARY`, `TECHNICIAN`, `ENGINEER`

* **Option 1 (Superclass + Subclass tables):**

* `EMP

* **Option 2 (Subclass-only tables - for Total/Disjoint):**

* `SEC

* **Option 3 (Single relation with 1 Type attribute):**

* `EMP

* **Option 4 (Single relation with Multiple Type attributes - for Overlapping):**

* `EMP
